

Unidad 1 — Calidad de diseño y construcción

Calidad interna vs. Calidad externa

- **Calidad Externa:** Es lo que el usuario final percibe. Incluye la **eficacia** (que el software haga lo que debe), la **fiabilidad** (que no falle), la **usabilidad** y el **rendimiento**. Se mide con métricas de QA y experiencia de usuario.
Ejemplo: La app funciona y responde rápido
- **Calidad Interna:** Cómo está construido el código. Se refiere a la **legibilidad**, el **bajo acoplamiento**, la **alta cohesión** y la **testabilidad**. El usuario no la ve, pero el desarrollador la sufre o la disfruta.
Ejemplo: Código claro, modular y mantenible

Ejemplos:

Versión 1 — Mala calidad interna (pero funciona)

```
public class DescuentoService {  
    public double calcular(double total, String tipoCliente) {  
        if (tipoCliente.equals("NORMAL")) {  
            return total;  
        } else if (tipoCliente.equals("VIP")) {  
            return total * 0.9;  
        } else if (tipoCliente.equals("EMPRESA")) {  
            return total * 0.8;  
        } else {  
            return total;  
        }  
    }  
}
```

Problemas de calidad interna:

- Strings hardcodeados
- Difícil de extender
- Uso excesivo de if-else
- Poca legibilidad

Calidad externa: Funciona correctamente para el usuario

Versión 2 — Buena calidad interna

```
public interface Descuento {
    double aplicar(double total);
}

public class DescuentoNormal implements Descuento {
    public double aplicar(double total) {
        return total;
    }
}

public class DescuentoVIP implements Descuento {
    public double aplicar(double total) {
        return total * 0.9;
    }
}

public class DescuentoEmpresa implements Descuento {
    public double aplicar(double total) {
        return total * 0.8;
    }
}

import java.util.Map;
public class DescuentoService {
    private Map<String, Descuento> estrategias;

    public DescuentoService() {
        estrategias = Map.of(
            "NORMAL", new DescuentoNormal(),
            "VIP", new DescuentoVIP(),
            "EMPRESA", new DescuentoEmpresa()
        );
    }

    public double calcular(double total, String tipoCliente) {
        return estrategias
            .getOrDefault(tipoCliente, new DescuentoNormal())
            .aplicar(total);
    }
}
```

Mejoras de calidad interna:

Código modular

Fácil de extender

Elimina if-else

Más legible

Calidad externa: El resultado es el mismo para el usuario

Conclusión : Ambas versiones funcionan igual para el usuario (tienen la misma calidad externa), pero la segunda tiene mejor calidad interna, lo que facilita mantenimiento y escalabilidad.

Deuda técnica y mantenibilidad.

- **Deuda Técnica:** Metáfora de Ward Cunningham. Tomar un "atajo" (código sucio o mal diseñado) para entregar rápido genera una deuda. Los "intereses" de esa deuda son el tiempo extra que tardarás en el futuro en agregar funcionalidades debido a la mala base. Osea es el costo futuro que pagamos por tomar atajos hoy.

Ejemplo:

- no escribir tests
- copiar código
- no refactorizar
- código confuso

Consecuencia:

- cambios más lentos
 - más bugs
 - mayor costo de mantenimiento
- **Mantenibilidad:** La capacidad del software para ser modificado (corregir errores, mejorar rendimiento o adaptar a cambios) con el menor esfuerzo y riesgo posible.

Capacidad de un sistema de:

- entenderse
- modificarse
- extenderse
- corregirse

Code Smells frecuentes y costo del cambio.

Concepto creado por **Martin Fowler**.

Un **code smell** es una señal de que algo en el diseño del código podría estar mal.

No es un bug, pero **indica posible problema de diseño**.

En un código con muchos *smells*, el costo del cambio crece de forma **exponencial**.

Smells frecuentes:

A. Método Largo (Long Method)

- **Descripción:** Funciones o métodos que intentan realizar demasiadas tareas simultáneamente.
- **Problema:** Son difíciles de leer, entender y, fundamentalmente, difíciles de testear de forma aislada.
- **Impacto:** El desarrollador "sufre" el código al intentar rastrear la lógica interna.

B. Clase Gigante (Large Class)

- **Descripción:** Clases que acumulan una cantidad excesiva de responsabilidades y líneas de código.
- **Violación Técnica:** Representa una violación directa al **Single Responsibility Principle** (Principio de Responsabilidad Única).

C. Obsesión por Primitivos (Primitive Obsession)

- **Descripción:** Tendencia a usar tipos de datos básicos (como `String`, `int` o `double`) para representar conceptos de dominio complejos.
- **Ejemplo:** Utilizar un simple `String` para manejar un Email en lugar de crear una clase `Email` que contenga su propia lógica de validación.

D. Demasiados Parámetros (Long Parameter List)

- **Descripción:** Métodos que requieren una lista extensa de argumentos (generalmente 5 o más) para ejecutarse.
- **Solución (Parameter Object):** Esta técnica de refactoring agrupa parámetros relacionados en una única clase u objeto.
- **Ventaja:** El método recibe un solo objeto estructurado en lugar de múltiples valores sueltos, mejorando la legibilidad.

E. Código Duplicado (Duplicated Code)

- **Descripción:** Considerado el "enemigo número uno" de la calidad interna.
- **Principio Violado:** Ataca el principio **DRY** (*Don't Repeat Yourself* - No te repitas).
- **Riesgo de Mantenimiento:** Si una regla de negocio cambia, el desarrollador debe buscar y modificar dicha lógica en múltiples lugares, lo que genera inconsistencias y errores.